

高可用设计

UPM 架构与高可用性设计文档

1. UPM 架构概述

UPM 系统由两大核心部分组成：

1.1 UPM Control

UPM 的核心控制平台，负责提供 API 接口，采用工作流方式管理 Kubernetes API 中的资源对象（CR）。

- 架构设计：**基于微服务架构，可快速扩展，满足多种数据库和中间件的运维需求。
- 功能定位：**通过模块化设计，支持不同类型数据库及中间件的运维工作流。

1.2 UPM Engine

UPM 的执行引擎，基于 Kubernetes Operator 框架开发，由两个自主研发的 Operator 构成：

- 功能定位：**专注于通用有状态服务的自动化运维及数据应用的高级操作。
- 架构特色：**采用通用工作负载管理设计，支持多种数据库类型和多种集群架构。

2. UPM Control 模块详解

UPM Control 基于 Spring Cloud 框架开发，模块化设计如下表所示：

序号	模块名称	描述
1	upm-control-gateway	Gateway API 模块，负责接收外部请求并路由到相应微服务模块。
2	upm-control-auth	Auth 模块，提供用户身份验证和权限管理功能。
3	upm-control-resource	Resource 模块，管理项目、集群、节点、存储类、软件等系统资源。
4	upm-control-user	User 模块，负责用户信息管理。
5	upm-control-operatorlog	OperatorLog 模块，记录系统操作日志，用于追踪和审计。
6	upm-control-mysql-ms	提供 MySQL 数据库完整运维 workflow 控制和管理功能。
7	upm-control-postgresql-ms	提供 PostgreSQL 数据库完整运维 workflow 控制和管理功能。
8	upm-control-redis-ms	提供 Redis 缓存完整运维 workflow 控制和管理功能。
9	upm-control-redis-cluster-ms	提供 Redis 集群缓存运维 workflow 控制和管理功能。
10	upm-control-kafka-ms	提供 Kafka 事件流完整运维 workflow 控制和管理功能。
11	upm-control-zookeeper-ms	提供 Zookeeper 服务发现完整运维 workflow 控制和管理功能。
12	upm-control-elasticsearch-ms	提供 Elasticsearch 搜索引擎完整运维 workflow 控制和管理功能。

每个模块使用 Kubernetes Deployment 部署，支持多副本模式以保证高可用性。

2.1 高可用支持

- **服务发现与故障切换：**基于 Nacos 实现，部署为 Cluster 模式，确保服务的高可用性。
- **多区域设计：**
 - **Region 分布：**多个区域（Region 1、Region 2），每个区域内有多个可用区（AZ）。
 - **数据同步：**同区内通过 Distro 协议实现一致性，跨区使用 Nacos-Sync 工具实现最终一致性。
 - **故障恢复：**通过心跳检测和自动故障切换机制，确保节点故障后其他节点接管任务。

3. UPM Engine高可用设计

UPM Engine 使用Operator模式进行开发，基于Leader Election 机制保证，controller的高可用。以下是具体高可用设计内容

3.1设计目标

1. 确保 Operator 运行的高可用性，在任何单节点故障情况下能够快速恢复。
2. 通过 Leader Election 机制，保证同一时刻只有一个副本负责处理资源事件，避免竞争和冲突。
3. 提供快速的故障切换（Failover）能力，保证 Operator 的稳定运行。

代码块

```
1 sequenceDiagram
2     participant Replica1 as Operator 副本 1
3     participant Replica2 as Operator 副本 2
4     participant Replica3 as Operator 副本 3
5     participant Lease as Kubernetes Lease
6
7     Replica1->>Lease: 尝试获取 Lease
8     Replica2->>Lease: 尝试获取 Lease
9     Replica3->>Lease: 尝试获取 Lease
10    Lease-->>Replica1: Lease 获取成功
11    Replica1-->>Replica2: Lease 被占用，进入 Standby
12    Replica1-->>Replica3: Lease 被占用，进入 Standby
13    Replica1->>Lease: 定期刷新租约
14    Note over Replica2,Replica3: 持续监听 Lease 状态
15
16    Replica1--xLease: Lease 超时 (Leader 故障)
17    Lease-->>Replica2: Lease 可用
18    Replica2->>Lease: 尝试获取 Lease
19    Replica3->>Lease: 尝试获取 Lease
20    Lease-->>Replica2: Lease 获取成功
21    Replica2->>Replica3: 通知进入 Standby
22    Replica2->>Lease: 开始刷新租约
```

3.2多副本部署

- **部署模式：**Operator 使用 Kubernetes Deployment 部署，支持多副本（建议 2 个及以上）。
- **调度策略：**通过 Anti-Affinity 确保 Pod 分布在不同节点。
- **Pod Disruption Budget：**限制同时被驱逐的 Pod 数量，避免服务中断。

3.3 Leader Election 机制

通过 Kubernetes ConfigMap 或 Lease 对象实现 Leader Election，确保同一时刻仅一个副本管理资源。

实现方式：

1. 启动阶段：多个副本争夺锁，获锁者为 Leader，其他副本进入 Standby 状态。
2. 故障切换：Leader 故障时，Standby 副本争夺锁并接管任务，切换时间为数秒。

参数调优：

- **Lease Duration**：租约时间（建议 15~30 秒）。
- **Retry Period**：争夺锁的间隔时间（默认 2 秒）。
- **Renew Deadline**：Leader 刷新锁的时间（通常为租约时间的 2/3）。

3.4故障切换机制

3.4.1 故障检测

- Kubernetes 的 Lease 对象未被 Leader 刷新时，Standby 副本会检测到锁超时并触发抢锁。
- 默认情况下，锁超时时间与检测间隔时间影响切换速度。

3.4.2 锁竞争

- Standby 副本通过抢占 Lease 对象的 `spec.holderIdentity` 字段成为新的 Leader。
- Kubernetes 确保只有一个副本能成功更新 Lease。

3.4.3 状态恢复

- 新的 Leader 副本启动后：
 - 检查所有资源的当前状态。
 - 根据实际状态与期望状态的差异，重新同步资源。
-

4. 调度高可用设计

在 Kubernetes 使用本地存储时，UPM 通过调度策略确保主从节点分布在不同物理节点上，提升容灾能力。

4.1 核心调度工具

- **Affinity**：指定 Pod 倾向于调度到特定节点。
- **Pod Anti-Affinity**：避免 Pod 部署在同一拓扑域，例如同一个主机组。

4.2 主机层级标签设计

在物理主机层面，使用 `hostgroup` 标签标识节点的分组：

- 示例：`hostgroup=alpha` 或 `hostgroup=beta`
此标签使调度器能感知主机组分布关系，实现跨主机组的高可用策略。

4.3 高可用策略

(1) 主机高可用

通过 Pod Anti-Affinity 和 `kubernetes.io/hostname` 键，确保主从节点分布在不同主机。

- **特点：**简单易实现，适用于快速规避单节点故障的场景。
- **局限：**无法规避主机组级别的故障。

(2) 主机组高可用

通过 Pod Anti-Affinity 和 `hostgroup` 标签，确保主从节点分布在不同主机组。

- **特点：**适合容灾要求高的场景，能规避主机组级别故障。
- **局限：**对资源调度灵活性要求高，需充裕的可用资源。

4.4高可用与故障恢复

- **健康检查：**通过 liveness/readiness probe 定期监控 Pod 的运行状态，确保节点故障能够被及时发现。
 - **自动重调度：**当节点不可用时，利用 Kubernetes 的重调度机制将 Pod 部署到其他满足条件的节点。
 - **故障切换：**在主节点故障时，从节点自动接管主节点职责，确保服务不中断。
-

5. 总结

UPM 系统通过模块化设计和高可用策略，提供强大的运维能力和稳定的运行保障：

1. **多模块支持：**涵盖 MySQL、Redis、Kafka 等多种数据库和中间件服务。
2. **调度高可用性：**基于 Affinity、Pod Anti-Affinity 和 `hostgroup` 标签实现主从节点隔离。
3. **Operator 高可用性：**多副本部署与 Leader Election 确保故障切换快速响应。

以上设计为复杂业务场景提供了稳定、高效的基础保障。